



Linux for Beginners

A Practical Guide

Chapter 9 of 12

Shell Scripting: Automating Tasks with Bash

Contents

9.1	What Is a Shell Script and Why Write One?
9.2	Your First Script: The Shebang and Hello World
9.3	Variables and User Input
9.4	Command Substitution and Arithmetic
9.5	Conditionals: if, elif, else
9.6	Test Expressions and Comparisons
9.7	Loops: for, while, and until
9.8	Functions
9.9	Handling Arguments and Special Variables
9.10	Exit Codes and Error Handling
9.11	Working with Files in Scripts
9.12	Practical Scripts: Real-World Examples
9.13	Essential Reference: Quick Lookup
9.14	Chapter Summary

Chapter 9

Shell Scripting: Automating Tasks with Bash

9.1 What Is a Shell Script and Why Write One?

A **shell script** is a plain text file containing a sequence of shell commands. When you run it, Bash executes each command in order — exactly as if you had typed them one by one in the terminal. But unlike typing commands manually, a script can be saved, reused, shared, scheduled, and parameterised.

Shell scripting is the glue of Linux system administration. Backups, deployments, log rotation, health checks, data processing, user management — virtually every repetitive task on a Linux system is a candidate for automation. Once you can write scripts, you stop doing the same thing twice.

When to write a script

- You've typed the same sequence of commands more than twice
 - A task needs to run automatically on a schedule (cron jobs)
 - You want to apply the same operation to many files or servers
 - You need to make decisions based on system state (disk full? service down?)
 - You want a repeatable, documented process that others can run
-

9.2 Your First Script: The Shebang and Hello World

Every Bash script starts with a **shebang line** — the first line of the file that tells the operating system which interpreter to use.

```
$ nano hello.sh

#!/bin/bash
# This is a comment – Bash ignores lines starting with #

echo "Hello, Linux!"
echo "Today is: $(date)"
echo "You are logged in as: $USER"
```

bash

Making the script executable and running it

```
# Save the file, then make it executable:
$ chmod +x hello.sh

# Run it:
$ ./hello.sh
Hello, Linux!
Today is: Mon Jun 10 10:42:01 UTC 2024
You are logged in as: ashik

# Alternatively, run it explicitly with bash:
$ bash hello.sh
```

TIP

The `./` before the script name tells Bash to look in the current directory. Without it, Bash only searches directories in your `PATH`. Always use `chmod +x` once, then run with `./scriptname.sh`

9.3 Variables and User Input

Variables store values you want to reuse. In Bash, you assign without spaces around `=` and read with a `$` prefix.

Defining and using variables

```
#!/bin/bash

# Assign a variable (no spaces around =):
name="Ashik"
age=25
greeting="Hello"

# Use variables with $:
echo "$greeting, $name!"
echo "You are $age years old."

# Wrap in braces when needed (e.g. joining with text):
file="report"
echo "Saving to ${file}_2024.txt"
```

Reading user input

```
#!/bin/bash

# Prompt the user for input:
echo "What is your name?"
read username
echo "Hello, $username!"

# Prompt on the same line:
read -p "Enter your name: " username
echo "Hello, $username!"

# Silent input (for passwords):
read -sp "Enter password: " password
echo
echo "Password stored (length: ${#password} chars)"
```

Environment variables

Bash automatically provides several useful variables:

Variable	Contains
\$USER	Your username
\$HOME	Your home directory path
\$PWD	Current working directory
\$PATH	Colon-separated list of directories searched for commands
\$SHELL	Path to the current shell
\$HOSTNAME	The machine's hostname
\$RANDOM	A random integer between 0 and 32767
\$SECONDS	Number of seconds since the script started

9.4 Command Substitution and Arithmetic

Command substitution

Command substitution lets you use the output of a command as a value — inside a variable or directly in a string:

```
#!/bin/bash

# Store command output in a variable:
current_date=$(date +%Y-%m-%d)
file_count=$(ls | wc -l)
disk_usage=$(df -h / | tail -1 | awk '{print $5}')

echo "Date: $current_date"
echo "Files in current dir: $file_count"
echo "Root disk usage: $disk_usage"

# Use it directly in a string:
echo "Hostname is: $(hostname)"
echo "Uptime: $(uptime -p)"
```

Arithmetic

```
#!/bin/bash

# Use $(( )) for integer arithmetic:
a=10
b=3

echo "Sum:          $((a + b))"
echo "Difference:   $((a - b))"
echo "Product:      $((a * b))"
echo "Quotient:     $((a / b))"    # integer division
echo "Remainder:    $((a % b))"

# Increment a counter:
counter=0
((counter++))
echo "Counter: $counter"

# For floating point, use bc:
result=$(echo "scale=2; 10 / 3" | bc)
echo "10 / 3 = $result"
```

9.5 Conditionals: if, elif, else

Conditionals let your script make decisions based on conditions. The structure always ends with `fi` (if backwards).

```
#!/bin/bash

# Basic if / else:
age=20

if [ $age -ge 18 ]; then
    echo "Adult"
else
    echo "Minor"
fi

# if / elif / else:
read -p "Enter a number: " num

if [ $num -gt 0 ]; then
    echo "Positive"
elif [ $num -lt 0 ]; then
    echo "Negative"
else
    echo "Zero"
fi

# Check if a command succeeded:
if ping -c 1 google.com &>/dev/null; then
    echo "Internet is reachable"
else
    echo "No internet connection"
fi
```

9.6 Test Expressions and Comparisons

The `[ ]` and `[[ ]]` constructs evaluate conditions. `[[ ]]` is the modern, more powerful version — prefer it in scripts.

Numeric comparisons

Expression	Meaning	Example
<code>-eq</code>	Equal to	<code>[\$a -eq \$b]</code>
<code>-ne</code>	Not equal to	<code>[\$a -ne \$b]</code>
<code>-gt</code>	Greater than	<code>[\$a -gt 10]</code>
<code>-ge</code>	Greater than or equal	<code>[\$a -ge 10]</code>

Expression	Meaning	Example
-lt	Less than	[\$a -lt 10]
-le	Less than or equal	[\$a -le 10]

String comparisons

Expression	Meaning	Example
= or ==	Equal	[[\$s == "hello"]]
!=	Not equal	[[\$s != "hello"]]
-z	Empty string	[[-z "\$s"]]
-n	Non-empty string	[[-n "\$s"]]
<	Less than (alpha)	[[\$a < \$b]]
>	Greater (alpha)	[[\$a > \$b]]

File and directory tests

Expression	True if...
-f file	file exists and is a regular file
-d dir	dir exists and is a directory
-e path	path exists (file or directory)
-r file	file is readable
-w file	file is writable
-x file	file is executable
-s file	file exists and has size > 0
-L file	file is a symbolic link


```
#!/bin/bash

# File test examples:
if [[ -f /etc/hosts ]]; then
    echo "/etc/hosts exists"
fi

if [[ ! -d ~/backups ]]; then
    mkdir ~/backups
    echo "Created backups directory"
fi

# Combine conditions with && (AND) and || (OR):
if [[ -f config.txt && -r config.txt ]]; then
    echo "Config file is readable"
fi
```

9.7 Loops: for, while, and until

for loop

```
#!/bin/bash

# Loop over a list of values:
for fruit in apple banana cherry; do
    echo "Fruit: $fruit"
done

# Loop over a range of numbers:
for i in {1..5}; do
    echo "Number: $i"
done

# Loop with a step:
for i in {0..20..5}; do
    echo "$i"
done

# Loop over files matching a pattern:
for file in *.txt; do
    echo "Processing: $file"
    wc -l "$file"
done

# C-style for loop:
for ((i=0; i<5; i++)); do
    echo "i = $i"
done
```

while loop

```
#!/bin/bash

# Loop while a condition is true:
counter=1
while [ $counter -le 5 ]; do
    echo "Counter: $counter"
    ((counter++))
done

# Read a file line by line:
while IFS= read -r line; do
    echo "Line: $line"
done < /etc/hosts

# Infinite loop with break:
while true; do
    read -p "Type 'quit' to exit: " input
    if [[ $input == "quit" ]]; then
        break
    fi
    echo "You typed: $input"
done
```

bash

until loop

```
#!/bin/bash

# until runs while the condition is FALSE (opposite of while):
count=1
until [ $count -gt 5 ]; do
    echo "Count: $count"
    ((count++))
done
```

bash

9.8 Functions

Functions let you group commands under a name and call them repeatedly. They make scripts readable, maintainable, and reusable.

```
#!/bin/bash

# Define a function:
greet() {
    echo "Hello, $1!"          # $1 is the first argument
    echo "Welcome to Linux."
}

# Call the function:
greet "Ashik"
greet "Alice"

# Function with a return value (via exit code):
is_even() {
    if (( $1 % 2 == 0 )); then
        return 0              # 0 = success/true in bash
    else
        return 1              # non-zero = failure/false
    fi
}

if is_even 4; then
    echo "4 is even"
fi

# Function returning a string (via echo + command substitution):
get_date() {
    echo $(date +%Y-%m-%d)
}

today=$(get_date)
echo "Today: $today"
bash
```

TIP

Define functions before you call them — Bash reads scripts top to bottom. A common pattern is to define all functions at the top, then call them at the bottom.

9.9 Handling Arguments and Special Variables

Scripts can accept arguments from the command line, making them flexible and reusable without editing the script itself.

```
#!/bin/bash
# Run as: ./backup.sh /home/ashik /backup/dest

# Special argument variables:
echo "Script name:      $0"
echo "First argument:   $1"
echo "Second argument:  $2"
echo "All arguments:    @"
echo "Number of args:   $#"
```

Variable	Meaning
\$0	Name of the script itself
\$1 – \$9	Positional arguments 1 through 9
\$@	All arguments as separate words
\$*	All arguments as a single word
\$#	Number of arguments passed
\$?	Exit code of the last command (0 = success)
\$\$	PID of the current script
\$_	PID of the last background command

Validating arguments

```
#!/bin/bash

# Check that the right number of arguments were provided:
if [[ $# -ne 2 ]]; then
    echo "Usage: $0 <source_dir> <dest_dir>"
    exit 1
fi

source_dir=$1
dest_dir=$2

if [[ ! -d $source_dir ]]; then
    echo "Error: $source_dir is not a directory"
    exit 1
fi

echo "Backing up $source_dir to $dest_dir"
```

9.10 Exit Codes and Error Handling

Every command in Linux returns an **exit code** when it finishes. **0** means success. Any other number means failure. Your scripts should check exit codes and handle errors gracefully.

```
#!/bin/bash

# Check the exit code of the previous command:
ls /nonexistent
echo "Exit code: $?" # will print a non-zero number

# Exit early on failure:
mkdir /tmp/mydir
if [[ $? -ne 0 ]]; then
    echo "Failed to create directory"
    exit 1
fi

# Cleaner: use set -e to exit on any error:
set -e

# Also useful:
set -u # treat unset variables as errors
set -o pipefail # catch errors in pipelines

# The idiomatic bash 'strict mode' header:
set -euo pipefail

# Run cleanup on exit using a trap:
cleanup() {
    echo "Cleaning up temp files..."
    rm -f /tmp/mytemp_$$
}
trap cleanup EXIT

# Now any exit (normal or error) will call cleanup
```

TIP

Always add 'set -euo pipefail' near the top of serious scripts. It makes your script exit immediately on errors instead of silently continuing with broken state — catching bugs that would otherwise be very hard to diagnose.

9.11 Working with Files in Scripts

Most real scripts read, write, or process files. Here are the most useful patterns:

```
#!/bin/bash

# Write to a file:
echo "Log started: $(date)" > script.log

# Append to a file:
echo "Step 1 done" >> script.log

# Read a file line by line:
while IFS= read -r line; do
    echo "Processing: $line"
done < input.txt

# Check if file is empty:
if [[ ! -s data.csv ]]; then
    echo "Error: data.csv is empty"
    exit 1
fi

# Create a temp file safely:
tmpfile=$(mktemp /tmp/myscript.XXXXXX)
echo "Using temp file: $tmpfile"
# Always clean up:
trap "rm -f $tmpfile" EXIT

# Redirect stdout and stderr to a log file:
exec >>(tee -a script.log) 2>&1
echo "This goes to screen AND script.log"
```

bash

9.12 Practical Scripts: Real-World Examples

Script 1: Automated backup with timestamp

```
#!/bin/bash
set -euo pipefail

SOURCE=$1
BACKUP_DIR="$HOME/backups"
DATE=$(date +%Y%m%d_%H%M%S)
DEST="$BACKUP_DIR/backup_$DATE.tar.gz"

# Validate input
if [[ $# -ne 1 ]]; then
    echo "Usage: $0 <directory_to_backup>"
    exit 1
fi

if [[ ! -d $SOURCE ]]; then
    echo "Error: $SOURCE is not a directory"
    exit 1
fi

# Create backup dir if needed
mkdir -p "$BACKUP_DIR"

# Create compressed archive
tar -czf "$DEST" "$SOURCE"
echo "Backup created: $DEST"
echo "Size: $(du -sh $DEST | cut -f1)"
```

Script 2: System health check


```
#!/bin/bash
set -euo pipefail

echo "==== System Health Check: $(date) ====="

# CPU load
load=$(uptime | awk -F'load average:' '{print $2}' | cut -d, -f1 | tr -d ' ')
echo "Load average (1min): $load"

# Memory usage
mem_used=$(free | awk '/Mem/{printf "%.0f", $3/$2*100}')
echo "Memory used: ${mem_used}%"

# Disk usage
disk_used=$(df / | tail -1 | awk '{print $5}')
echo "Root disk used: $disk_used"

# Warn if disk > 80%
disk_num=${disk_used%\%}
if [[ $disk_num -gt 80 ]]; then
    echo "WARNING: Disk usage is above 80%!"
fi

echo "==== Done ====="
```

Script 3: Batch file rename

```
#!/bin/bash
# Rename all .txt files to .bak in current directory

count=0
for file in *.txt; do
    if [[ -f $file ]]; then
        newname="${file%.txt}.bak"
        mv "$file" "$newname"
        echo "Renamed: $file -> $newname"
        ((count++))
    fi
done

echo "Done. Renamed $count files."
```

9.13 Essential Reference: Quick Lookup

Syntax / Command	What It Does
<code>#!/bin/bash</code>	Shebang — declare bash as the interpreter
<code>chmod +x script.sh</code>	Make script executable
<code>./script.sh</code>	Run a script in the current directory
<code>name="value"</code>	Assign a variable (no spaces around =)
<code>\$name</code> or <code>\${name}</code>	Read a variable
<code>read -p 'Prompt: ' var</code>	Read user input into a variable
<code>\$(command)</code>	Command substitution — use output as a value
<code>\$((a + b))</code>	Arithmetic expression
<code>if [[cond]]; then ... fi</code>	Conditional block
<code>-eq -ne -gt -ge -lt -le</code>	Numeric comparison operators
<code>== != -z -n</code>	String comparison operators
<code>-f -d -e -r -w -x -s</code>	File test operators
<code>for x in list; do ... done</code>	For loop over a list
<code>for i in {1..10}; do</code>	For loop over a range
<code>while [[cond]]; do</code>	While loop
<code>until [[cond]]; do</code>	Until loop (runs while condition is false)
<code>break / continue</code>	Exit loop / skip to next iteration
<code>funcname() { ... }</code>	Define a function
<code>\$0 \$1 \$2 ... @\$#</code>	Script name, arguments, all args, arg count
<code>\$?</code>	Exit code of last command (0 = success)
<code>\$\$</code>	PID of the current script
<code>set -euo pipefail</code>	Strict mode — exit on errors, unset vars, pipe fails
<code>trap 'cmd' EXIT</code>	Run cmd when script exits (cleanup)
<code>mktemp /tmp/name.XXXXX</code>	Create a safe temporary file

9.14 Chapter Summary

- ✓ A shell script is a plain text file of bash commands. Start every script with `#!/bin/bash`, make it executable with `chmod +x`, and run it with `./scriptname.sh`.
- ✓ Variables are assigned with `name=value` (no spaces). Read them with `$name`. Use `read -p` to accept user input.
- ✓ Command substitution `$(command)` captures command output as a value. Arithmetic uses `$( ( ))`.
- ✓ Conditionals use `if [[ condition ]]; then ... elif ... else ... fi`. Prefer `[[ ]]` over `[ ]` in modern scripts.
- ✓ Three comparison families: numeric (`-eq -gt -lt`), string (`== != -z -n`), and file (`-f -d -e -r -w -x`).
- ✓ Loops: `for` iterates over lists or ranges; `while` runs while true; `until` runs while false. Use `break` and `continue` to control flow.
- ✓ Functions group reusable logic. Define before calling. Arguments inside functions are `$1`, `$2`, etc. Return values via `echo + command substitution`.
- ✓ Script arguments: `$1–$9` for positional args, `$@` for all, `$#` for count. Always validate arguments before using them.
- ✓ `$?` holds the exit code of the last command. Use `set -euo pipefail` at the top of serious scripts to catch errors immediately.
- ✓ `trap 'cleanup' EXIT` runs cleanup code whenever the script exits — normal or error. Always use it when creating temp files.

What's Coming in Chapter 10

With scripting mastered, Chapter 10 goes deeper into the Linux filesystem itself — how storage is organised, managed, and extended. You'll learn:

- How Linux represents disks, partitions, and filesystems
- Mounting and unmounting storage devices with `mount`
- The `/etc/fstab` file — automatic mounts at boot
- Creating and formatting partitions with `fdisk` and `mkfs`
- Logical Volume Management (LVM) — flexible disk management
- Symbolic links and hard links — the difference and when to use each
- Inodes, file descriptors, and how Linux really stores files

Chapter 10 is where the filesystem stops being a black box.

Linux for Beginners — Chapter 9 of 12

Author

Ashik A

github.com/ashihh07

This guide was created, organized, refined, and designed by Ashik A using GitHub documentation, industry best practices, publicly available learning resources, and AI-assisted research.
